🧠 Diseño de Sistemas - Preguntas Frecuentes en Entrevistas

Este documento recopila preguntas comunes sobre **Diseño de Sistemas** utilizadas en entrevistas técnicas de nivel medio a senior. Incluye explicaciones, ejemplos visuales y conceptos clave que te ayudarán a estructurar soluciones escalables y robustas.

🔹 **Cada pregunta está explicada directamente en este archivo.**
Puedes navegar usando el índice para ir a la sección que más te interese.

📌 Índice de Preguntas

1. ¿Qué es el diseño de sistemas?
2. ¿Qué pasos sigues para diseñar un sistema desde cero?
3. ¿Qué es la escalabilidad horizontal vs. vertical?
4. ¿Qué es la tolerancia a fallos?
5. ¿Qué es un balanceador de carga?
6. ¿Qué es un CDN y para qué se usa?
7. ¿Qué es una base de datos relacional vs. no relacional?
8. ¿Qué es la partición o "sharding"?
9. ¿Qué es la caché y cómo mejora el rendimiento?
10. ¿Qué es CAP Theorem?
11. ¿Qué es una cola de mensajes y por qué es útil?
12. ¿Qué métricas considerarías para evaluar el rendimiento de un sistema?
13. ¿Cómo diseñarías un sistema tipo URL Shortener como bit.ly?
14. ¿Cómo diseñarías un sistema de chat en tiempo real?
15. ¿Cómo garantizarías alta disponibilidad en un sistema?

---

1. ¿Qué es el diseño de sistemas?

Es el proceso de definir la arquitectura, componentes y datos de un sistema de software. Se enfoca en cómo los distintos módulos interactúan y escalan.

---

2. ¿Qué pasos sigues para diseñar un sistema desde cero?

1. Requisitos funcionales y no funcionales.
2. Estimación de tráfico y carga.
3. Diseño de alto nivel (componentes principales).
4. Diseño de base de datos.
5. Arquitectura detallada (API, colas, cache, etc.).

6. Consideraciones de escalabilidad, disponibilidad y resiliencia.
7. Seguridad y monitoreo.

---

3. ¿Qué es la escalabilidad horizontal vs. vertical?

- **Horizontal**: Añadir más máquinas.
- **Vertical**: Añadir más recursos a una sola máquina.

💡 *Escalar horizontalmente suele ser preferido por su flexibilidad.*

---

4. ¿Qué es la tolerancia a fallos?

Es la capacidad de un sistema de seguir funcionando incluso si una parte falla. Se logra con redundancia, replicación y manejo de errores.

---

5. ¿Qué es un balanceador de carga?

Es un componente que distribuye el tráfico entre múltiples servidores para evitar sobrecarga y mejorar disponibilidad.

---

6. ¿Qué es un CDN y para qué se usa?

Un **CDN (Content Delivery Network)** es una red de servidores distribuidos que entregan contenido estático (imágenes, videos, archivos) desde ubicaciones cercanas al usuario para mejorar la velocidad.

---

7. ¿Qué es una base de datos relacional vs. no relacional?

Tipo	Ejemplos	Características
Relacional	MySQL, PostgreSQL	Basadas en tablas, SQL
No Relacional	MongoDB, Redis	JSON, clave-valor, documentos

---

8. ¿Qué es la partición o "sharding"?

Es dividir una base de datos grande en partes más pequeñas llamadas "shards", distribuidas en distintos servidores para mejorar rendimiento y escalabilidad.

---

9. ¿Qué es la caché y cómo mejora el rendimiento?

La caché almacena datos en memoria para evitar accesos repetidos a bases de datos. Mejora la latencia y reduce la carga del sistema.

Ejemplo: Redis, Memcached.

---

10. ¿Qué es CAP Theorem?

Postula que en un sistema distribuido solo se pueden garantizar **dos de tres** propiedades al mismo tiempo:

- **C**onsistency (consistencia)
- **A**vailability (disponibilidad)
- **P**artition Tolerance (tolerancia a partición)

---

11. ¿Qué es una cola de mensajes y por qué es útil?

Es un sistema que permite comunicación asíncrona entre servicios. Se usa para desacoplar componentes y manejar picos de tráfico.

Ejemplo: RabbitMQ, Kafka, AWS SQS.

---

12. ¿Qué métricas considerarías para evaluar el rendimiento de un sistema?

- Latencia
- Throughput
- Tasa de errores
- Tiempo de respuesta promedio
- Uso de CPU/Memoria
- SLA/Uptime

---

13. ¿Cómo diseñarías un sistema tipo URL Shortener como bit.ly?

- Endpoint para generar la URL corta.
- Base de datos para mapear `<shortCode>` → `<originalURL>`.
- Algoritmo de hash o generador incremental.
- Caché para accesos frecuentes.
- Escalabilidad: separación de lectura/escritura.

---

14. ¿Cómo diseñarías un sistema de chat en tiempo real?

- WebSockets para comunicación bidireccional.
- Cola de mensajes para persistencia.
- Base de datos para historial.
- Escalabilidad horizontal del servidor de sockets.

---

15. ¿Cómo garantizarías alta disponibilidad en un sistema?

- Clusters de servidores redundantes.
- Balanceadores de carga.
- Réplicas de bases de datos.
- Monitorización y alertas.
- Backup automático y recuperación ante desastres.

📚 Recursos Adicionales

- [System Design Primer (GitHub)](https://github.com/donnemartin/system-design-primer)
- [Educative.io - Grokking the System Design Interview](https://www.educative.io/courses/grokking-the-system-design-interview)
- [High Scalability Blog](http://highscalability.com/)
- [Awesome Scalability](https://github.com/binhnguyennus/awesome-scalability)

🎥 **Videos recomendados**:

- [Design Netflix (Hussein Nasser)](https://www.youtube.com/watch?v=UlR8dANbQWI)
- [System Design Basics (Tech Dummies)](https://www.youtube.com/watch?v=UzLMhqg3_Wc)

Hecho con 🧠 por [@holasoymalva](https://holasoymalva.com)